



电子科技大学

University of Electronic Science and Technology of China

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W3D2 HTTP处理静态资源

Linux下HTTP Server的设计

实训目标：让V1.1真正返回一个网站

本次升级完成三件事：

1. 把请求URL安全映射到www目录中的文件
2. 根据扩展名返回正确的Content-Type和Content-Length
3. 分块发送文件，并保留事件管理、日志和fd清理闭环

最终成果：浏览器能够完整加载HTML、CSS、JavaScript、图片和favicon。



W3D2只新增静态文件层

V1.1已经具备：

- epoll 管理多个client_fd
- HTTP请求完整性判断、请求行和请求头解析
- 200 / 404响应、访问日志和系统日志

V1.2继续增加：

- 请求路径 → 本地文件 → MIME类型 → 文件响应

事件循环不重写，HTTP公共解析和日志模块继续复用。



一次静态资源请求的完整处理流程

E POLLIN 通知client_fd可读



recv()追加到该客户端独立缓冲区



请求完整 → 解析方法和目标路径



规范化路径 → 限定www根目录 → stat()



确定状态码、MIME和Content-Length



发送响应头 → 分块发送文件 → 日志 → 删除事件 → close()



URL路径映射到固定网站根目录

服务器启动时指定document root, 例如www/:

GET / → www/index.html

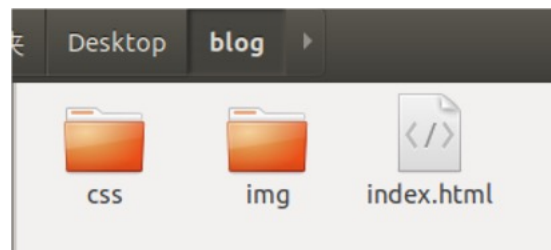
GET /index.html → www/index.html

GET /css/style.css → www/css/style.css

GET /js/app.js → www/js/app.js

GET /img/logo.jpeg → www/img/logo.jpeg

多级目录通过URL路径直接定位, 不需要opendir()遍历整个网站目录。



先规范化请求目标，再查找文件

请求行示例：GET /css/style.css?v=1 HTTP/1.1

1. 只处理路径部分，先去除?后面的查询参数
2. 将/映射为/index.html
3. 在最终路径中提取扩展名，并统一转为小写
4. GET进入静态资源处理；其他方法返回405和Allow: GET
5. 不要直接把整条请求行或未经检查的target拼到本地路径

输出结果：/css/style.css，而不是/css/style.css?v=1。



网站根目录是静态文件的安全边界

document root: 服务器允许公开的唯一目录, 例如/www/blog。

- 启动时使用realpath()得到根目录绝对路径
- 拒绝空字节和包含..的路径段
- 对存在的目标执行realpath(), 结果必须仍以根目录/开头
- 符号链接如果跳出根目录, 返回403
- 不存在的文件返回404, 不能回退读取服务器任意文件

必须测试: `curl --path-as-is http://127.0.0.1:9001/../../etc/passwd`



MIME告诉浏览器如何解释响应体

Content-Type由资源类型决定，而不是由HTTP状态码决定。

相同的一串字节，MIME不同，浏览器的处理方式也不同：

- text/html：解析成页面结构
- text/css：作为样式表加载
- text/javascript：作为脚本执行
- image/png：作为图片解码

实现时先得到规范化文件路径，再根据最终扩展名查MIME映射表。



常用静态资源MIME类型

1. 文本文件:

- text/plain: 纯文本文件
- text/html: HTML 文档
- text/css: CSS 样式表
- text/javascript: JavaScript 脚本

2. 图像文件:

- image/jpeg: JPEG 图像
- image/png: PNG 图像
- image/gif: GIF 图像
- image/svg+xml: SVG 图像

3. 音频文件:

- audio/mpeg: MP3 音频
- audio/ogg: Ogg Vorbis 音频

4. 视频文件:

- video/mp4: MP4 视频
- video/ogg: Ogg 视频

5. 应用程序文件:

- application/json: JSON 数据
- application/xml: XML 数据
- application/pdf: PDF 文档
- application/zip: ZIP 压缩文件

6. 二进制文件:

- application/octet-stream: 用于传输二进制数据, 如可执行文件或字节流

7. 其他:

- multipart/form-data: 用于表单提交, 包含键值对和文件上传
- application/x-www-form-urlencoded: 用于表单提交, 键值对编码为URL参数



文本资源显式使用UTF-8

推荐映射：

text/html; charset=utf-8 .html .htm

text/css; charset=utf-8 .css

text/javascript; charset=utf-8 .js

application/json; charset=utf-8 .json

image/jpeg / image/png / image/gif / image/vnd.microsoft.icon

未知扩展名使用application/octet-stream。

现代HTTP实现不应依赖“文本默认US-ASCII”，文本资源应显式写charset。

媒体类型名称通常使用小写；二进制资源不要添加charset。



stat()同时提供文件类型和文件大小

头文件: #include <sys/stat.h>

```
struct stat st;  
if (stat(file_path, &st) == -1) { /* 根据errno返回错误 */ }
```

st.st_mode: 判断目标是否为普通文件、目录或其他类型

st.st_size: 文件字节数, 直接用于响应Content-Length

stat()失败时:

- ENOENT / ENOTDIR → 404
- EACCES → 403
- 其他错误 → 500并写system.log

涉及符号链接安全时, 结合lstat()或realpath()检查最终目标。



静态服务器只发送允许公开的普通文件

S_ISREG(st.st_mode) → 可以发送, 返回200

S_ISDIR(st.st_mode) → 本次只允许/映射index.html; 其他目录返回403

S_ISLNK(st.st_mode) → 校验真实路径仍在document root内

其他文件类型 → 返回403

为什么不能直接发送所有路径?

- 目录不是HTTP响应体文件
- 设备、FIFO、socket可能阻塞或泄露系统信息
- 符号链接可能跳出网站根目录

按请求路径直接stat()即可; 本次无需opendir()和readdir()。



errno只在系统调用明确失败后读取

头文件: #include <errno.h>

1. 先检查open()、stat()、read()等函数的返回值
2. 只有返回值表示失败时, 才读取errno
3. perror("open")把说明和errno对应信息写到标准错误stderr
4. strerror(errno)返回错误字符串; 安全输出: printf("%s\n", strerror(errno));

Web服务器中的错误映射:

ENOENT / ENOTDIR → 404

EACCES → 403

其他I/O错误 → 500

成功调用后errno可能保留旧值, 因此不能脱离失败返回值单独判断errno。



正确处理open()失败

```
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(void) {
    int fd = open("www/missing.html",
0_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    close(fd);
    return 0;
}
```

```
root@yan934:/home/dir1# ./error.o
./error.o: No such file or directory
No such file or directory
root@yan934:/home/dir1#
```



响应头由文件元数据生成

HTTP/1.1 200 OK\r\n

Content-Type: text/css; charset=utf-8\r\n

Content-Length: 1260\r\n

Connection: close\r\n

\r\n

[1260字节文件内容]

Content-Length只计算响应体字节数，不包含状态行和响应头。

二进制文件必须按明确长度发送，不能使用strlen()判断文件长度。

发送完响应后统一删除事件并关闭client_fd。



固定大小缓冲区分块发送文件

错误做法

```
char *p = malloc(file_size);
read(file_fd, p, file_size);
send(client_fd, p, file_size, 0);
free(p);
```

文件越大，单个请求占用内存越多。

推荐做法

```
char buffer[8192];
while ((n = read(file_fd, buffer, sizeof(buffer))) > 0) {
    if (send_all(client_fd, buffer, (size_t)n) != 0)
        break;
}
close(file_fd);
```

内存占用与文件大小解耦；不要把二进制文件当作C字符串处理

一次send()不保证发送全部字节

send_all()必须维护已发送偏移:

```
sent = 0
```

```
while sent < length:
```

```
    n = send(fd, data + sent, length - sent, 0)
```

```
    n > 0    → sent += n
```

```
    errno=EINTR → 重试
```

```
    其他结果 → 失败并进入统一清理
```

基础版用于较小静态文件，可以采用阻塞发送。

进一步扩展非阻塞fd时，应保存待发送数据，并监听EPOLLOUT继续发送。



文件处理仍然嵌入原有事件生命周期

epoll_wait()



client_fd可读: 接收并累计HTTP请求



请求完整: serve_static_file(client_fd, target)



路径校验 → stat → open → 响应头 → 文件内容



access.log记录URL、MIME、状态、字节数和耗时



epoll_ctl(DEL) → close(client_fd)



本次实训只需要六个核心状态码

200 OK	普通文件成功发送
400 Bad Request	路径或请求格式非法
403 Forbidden	路径越界、无权限或不是普通文件
404 Not Found	文件或中间目录不存在
405 Method Not Allowed	不是GET；同时返回Allow: GET
500 Internal Server Error	服务器内部I/O错误

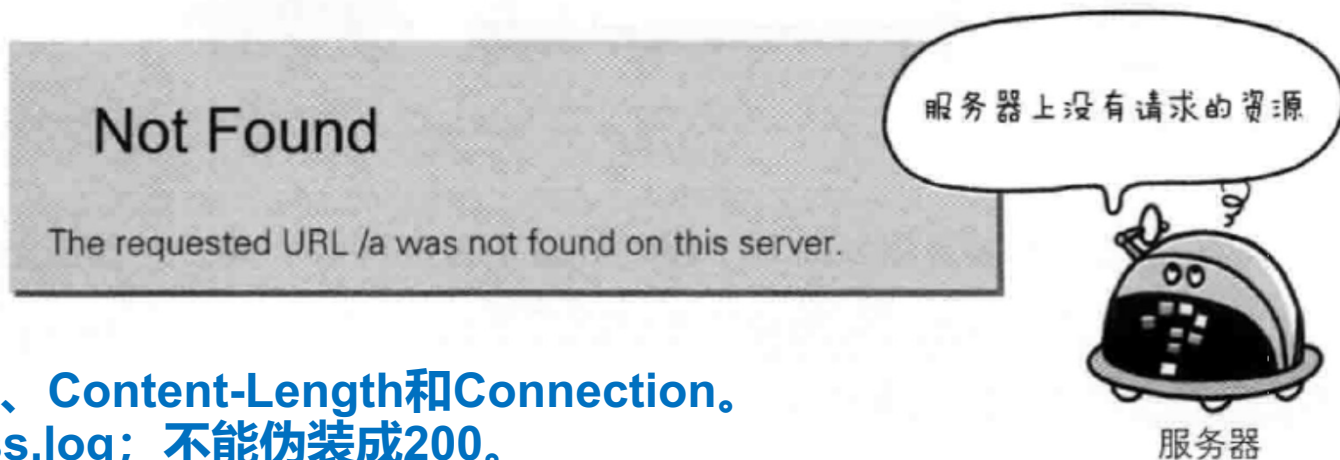
状态码、响应正文、访问日志必须表达同一个结果。

304缓存和206范围请求属于后续扩展，不纳入V1.2最低验收。



404表示请求的静态资源不存在

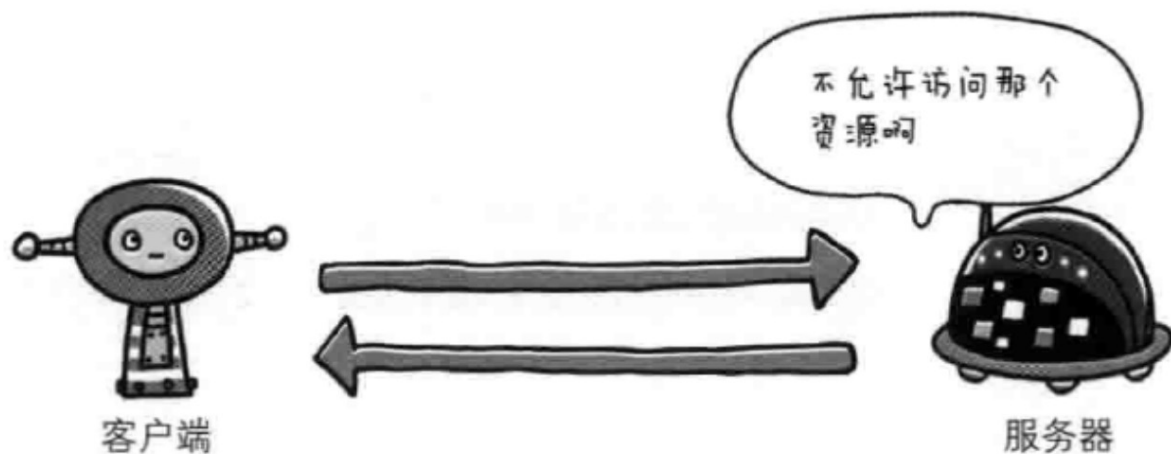
404 Not Found
ENOENT / ENOTDIR → 404



响应仍要包含正确Content-Type、Content-Length和Connection。
404是一次访问结果，写入access.log；不能伪装成200。



403表示服务器拒绝公开该目标



重点测试:

`curl --path-as-is`

`http://127.0.0.1:9001/../../etc/passwd`

预期: 403, 且www之外的文件绝不能被读取。

不要在响应正文中泄露服务器真实绝对路径。



日志和fd清理是响应的一部分

access.log: 一次HTTP访问一条记录

客户端IP:端口、方法、URL、MIME、状态码、响应字节数、耗时、发送结果

system.log: 服务器运行和内部错误

启动、退出、连接、断开、stat/open/read/send失败、fd和errno

统一清理顺序:

close(file_fd) → 删除client_fd事件 → close(client_fd) → 释放缓冲区

404首先是访问结果, 不应当作为服务器内部ERROR; 真正I/O异常才记录ERROR。



按可运行的小步骤完成V1.2

1. 建立www/目录并准备index.html、CSS、JS、图片和favicon
2. 先完成/和/index.html映射，浏览器能看到HTML
3. 增加查询参数去除、路径规范化和document root校验
4. 使用stat()判断文件并生成Content-Length
5. 根据扩展名返回MIME，使CSS、JS和图片正常加载
6. 使用固定缓冲区和send_all()发送文件
7. 补齐403、404、405、500和统一fd清理
8. 最后增加访问日志、系统日志和自动测试证据

每完成一步就运行curl，不要等全部代码写完后再调试。



验收必须同时证明功能、安全和清理

1) 功能证据

- 浏览器首页完整显示；HTML、CSS、JS、图片和favicon均为200
- curl -i /、/css/style.css、/img/logo.jpeg返回正确Content-Type和长度

2) 错误与安全证据

- /missing返回404；非GET返回405；/../etc/passwd被拒绝

3) 并发与清理证据

- 3个客户端连续访问互不影响
- 响应后删除事件并关闭client_fd；日志包含URL、状态码和字节数

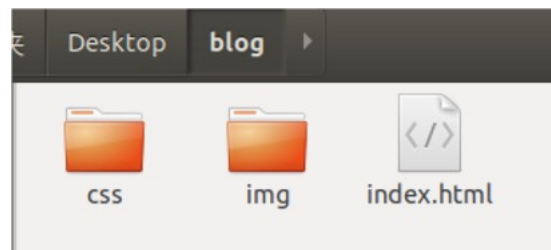
4) 代码说明：学生能指出路径校验、MIME映射、分块发送和统一清理的位置。

实训内容：完成mini_webserver V1.2

每名学生在W3D1 mini_webserver V1.1基础上升级：

- 建立www/博客目录，至少包含HTML、CSS、JS、GIF/JPEG和ICO
- GET /与GET /index.html均返回首页，多级目录资源可正常加载
- 正确设置Content-Type、Content-Length和Connection
- 固定缓冲区分块读取文件，并处理部分发送
- 限定document root，拒绝目录穿越和非普通文件
- 正确返回200、400、403、404、405、500
- 保留epoll事件模型、日志系统和统一fd清理

最终提交：可运行源代码、网站资源、测试命令及日志/终端验收证据。



学生本次实训前需提前准备

- 可运行的mini_webserver V1.1。
- www/目录及index.html、CSS、JavaScript、JPEG或GIF图片、favicon.ico。
- 两个终端：一个运行服务器，一个执行curl测试。
- 可用于路径穿越测试的命令：
 - curl -i http://127.0.0.1:9001/
 - curl -i http://127.0.0.1:9001/css/style.css
 - curl -i http://127.0.0.1:9001/missing
 - curl -i -X POST http://127.0.0.1:9001/
 - curl -i --path-as-is http://127.0.0.1:9001/../etc/passwd



运行测试建议

1. 正常资源

```
curl -i http://127.0.0.1:9001/
```

```
curl -i http://127.0.0.1:9001/index.html
```

```
curl -i http://127.0.0.1:9001/css/style.css
```

```
curl -s -D - -o /dev/null http://127.0.0.1:9001/img/logo.jpeg
```

最后一条命令仍然发送GET，-D -把响应头输出到终端，-o /dev/null 丢弃图片响应体，避免二进制内容直接显示在终端中。



2. 错误与安全

```
curl -i http://127.0.0.1:9001/missing
```

```
curl -i -X POST http://127.0.0.1:9001/
```

```
curl -i --path-as-is http://127.0.0.1:9001/../etc/passwd
```



3. 三客户端访问

在三个终端分别执行：

```
curl -i http://127.0.0.1:9001/
```

```
curl -i http://127.0.0.1:9001/css/style.css
```

```
curl -i http://127.0.0.1:9001/img/logo.jpeg
```

观察三个请求是否都能完成，服务器是否继续运行，日志是否出现三条对应访问记录。



常见问题与提示

现象	优先检查位置	提示
----	--------	----

首页能打开, CSS无效果	CSS URL、路径映射、MIME	在浏览器Network中查看CSS状态码和Content-Type
---------------	-------------------	-----------------------------------

图片只收到一部分	read()循环、send()返回值	不能假设一次send()发送全部数据
----------	--------------------	--------------------

请求/返回404	/到/index.html映射	先打印规范化后的URL和最终文件路径
----------	-----------------	--------------------

所有资源长度都不正确	Content-Length生成	文件长度来自st.st_size, 不是strlen()
------------	------------------	------------------------------

请求不存在文件导致服务器退出	stat/open错误分支	404是正常访问结果, 不能直接终止服务器进程
----------------	---------------	-------------------------

运行一段时间后无法建立连接	fd和客户端状态清理	检查所有分支是否删除事件、关闭fd、释放缓冲区
---------------	------------	-------------------------

路径穿越能够读取外部文件	规范化和根目录校验	realpath()后必须验证目标仍在document root内
--------------	-----------	-----------------------------------

非阻塞发送CPU占用很高	EAGAIN处理	不要忙等, 保存发送状态并等待可写事件
--------------	----------	---------------------



总结

V1.1解决的是“服务器怎样收到并解析HTTP请求”，V1.2解决的是“服务器怎样把请求安全地转换为正确的静态文件响应”。

记住五个关键词：**安全路径、文件元数据、正确MIME、可靠发送、统一清理**。这五个部分任何一个缺失，服务器都可能出现资源加载失败、安全越界、文件截断或fd泄漏。



思考

1. 为什么路径映射必须限制在document root内?
2. stat()、lstat()和realpath()在本项目中分别解决什么问题?
3. 为什么Content-Length必须与实际发送的响应体字节数一致?
4. 阻塞fd和非阻塞fd在处理部分发送时有什么差异?
5. 为什么404通常写访问日志, 而不是作为系统内部ERROR?

